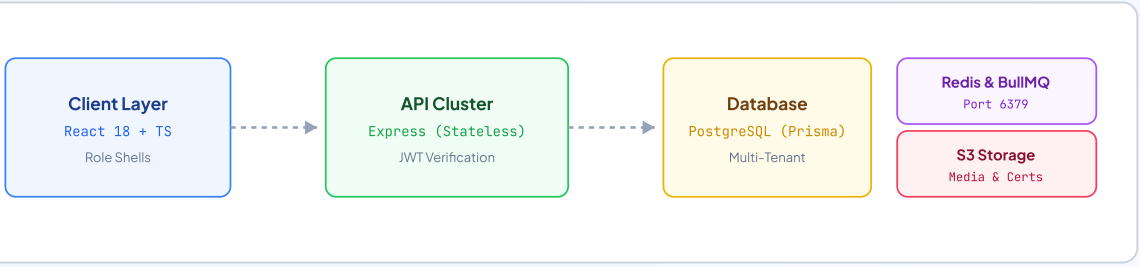
FORGE LOOM LMS

System Architecture Reference

Full Platform System Architecture & Infrastructure Guide

The engineering backbone for the FORGE Program (Winnovation). Comprehensive reference detailing the PERN stack, multi-tenant isolation, asynchronous score computation, Redis queues, and horizontal scaling.



STACK BASELINE
PostgreSQL + Express + React + Node.js (PERN)

CONCURRENCY TARGET
5,000 – 50,000 Concurrent Users

ASYNCHRONOUS TASKS
BullMQ Job Queue (Score Engine, PDF Certs, Notifs)

TENANCY MODEL
College-Scoped Row-Level Security

Winnovation Systems Engineering

October 2026 • Architectural Blueprint

1 Core Architectural Principles

The architecture of FORGE Loom is governed by five non-negotiable engineering principles designed to guarantee stability, horizontal scalability, and data integrity under high concurrent loads:

1. One Identity System, Distinct Role Shells

Identity

Every stakeholder (Student, Mentor, Trainer, HR, College Admin) logs into the exact same application. The moment authentication resolves, the app renders a dedicated dashboard shell specifically tailored for their role. Role is a first-class enum (`Role`) on the `User` record, not a fragile permission flag.

2. Strict Separation: Write-Heavy vs. Read-Heavy Paths

Performance

Attendance logging, video timestamp updates, and sprint milestone submissions are write-heavy, append-only streams. In contrast, the HR Talent Pool candidate search (filtering by skill, domain, and score) is read-heavy. These paths receive distinct indexing and caching treatment in PostgreSQL and Redis.

3. Derived Scoring Engine (Asynchronous Recomputation)

Calculation

A student's Builder Score is strictly computed—never manually typed in. It is derived from four weighted inputs: **Events (20%)**, **Projects (40%)**, **Mentor Reviews (30%)**, and **Team Collaboration (10%)**. Recomputations are dispatched asynchronously via BullMQ to avoid blocking HTTP request cycles.

4. Citadel Innovation Incubator as an Explicit State Machine

State Machine

Citadel is a formalized state machine bolted onto Student and Mentor data. Sprint cycle progression (Sprint 1, 2, 3), milestone submission states (`not_started` , `in_progress` , `submitted` , `reviewed` , `approved`), and investor unlock gates are explicitly modeled as discrete states.

5. Zero Synchronous Heavy Lifting in Request Cycles

Asynchronous

Certificate generation, notification fan-out, report exports, and score recomputation are processed via Redis-backed BullMQ queues. The API confirms receipt immediately, ensuring response times remain sub-50ms.

2 Builder Score Engine & Queue Architecture

The Builder Score is the central gamification and talent discovery metric. The flowchart below outlines the event-driven scoring pipeline:

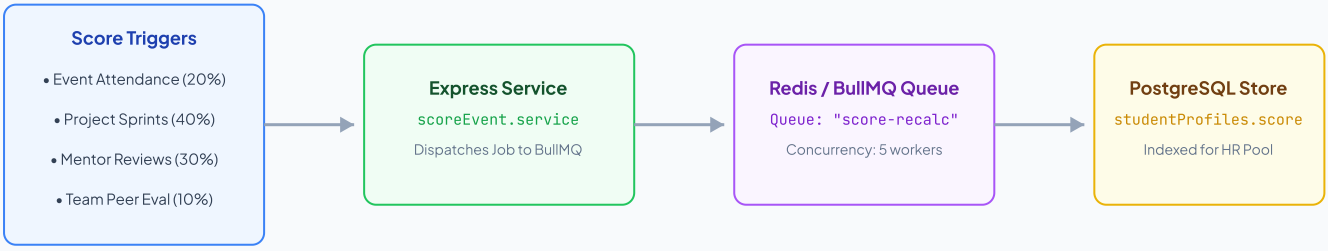


FIGURE 2.1: ASYNCHRONOUS SCORE CALCULATION PIPELINE VIA BULLMQ & REDIS

3 Multi-Tenant College Isolation Model

FORGE Loom enforces strict multi-tenancy at the data layer. College Admins, Students, Mentors, and Trainers belonging to **College 1** (Forge Institute of Technology) are completely partitioned from **College 2** (Riverside College of Engineering).

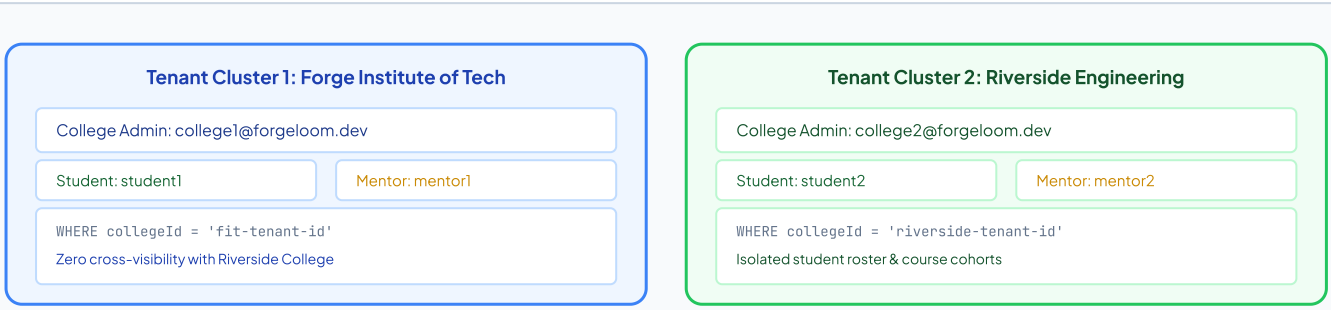


FIGURE 3.1: MULTI-TENANT DATA SCOPING ARCHITECTURE IN POSTGRESQL VIA PRISMA

Entity	Tenant Scoping Column	Isolation Rule
`User`	`collegeId (Nullable String)`	Null for global roles (ForgeAdmin, HR, Speaker). Populated for college-scoped users.
`StudentProfile`	`collegeId (String)`	Query filter guarantees student rosters only display to authorized campus administrators.
`Cohort`	`collegeId (String)`	Batch schedules and course access are locked to institutional cohorts.
`Team`	`collegeId (String)`	Citadel teams are formed within college boundaries to preserve cohort dynamics.

4 Production Deployment & Horizontal Scaling

To support the target scale of **50,000 concurrent active users**, FORGE Loom separates the stateless API nodes from the stateful database and queue tiers.

Stateless API Cluster Behind Load Balancer

Production Setup

- **Stateless Express API Nodes:** Multiple Node.js instances behind an AWS ALB or Cloudflare CDN. Any node can handle any request since sessions are stored in JWT tokens and Redis.
- **Database Connection Pooling:** Prisma Client with PgBouncer connection pooling to prevent PostgreSQL connection saturation under peak spike loads.
- **Redis Socket.IO Adapter:** Enables real-time sprint updates and instant notification broadcasts across all distributed API containers.
- **Object Storage CDN:** Student uploaded assignments, certificates, and video streaming assets are served directly through Cloudflare R2 / AWS S3 presigned URLs.

Repository & Architectural Verification

The system architecture is maintained in the official GitHub repository:

```
https://github.com/Winnovationforge/Forge-loom.git
```